Python 程序设计语言



第三部分 数据的容器（结构）

字符串、列表、元组、字典、集合

序列数据结构

- 数据结构是计算机存储、组织数据的方式。序列是Python中最基本的数据结构。
- 序列中的每个元素都分配一个数字即它的位置或索引，第一个索引是0，第二个索引是1，依此类推。序列都可以进行的操作包括索引，截取（切片），加，乘，成员检查。此外，Python已经内置确定序列的长度以及确定最大和最小的元素的方法。
- Python内置序列类型最常见的是列表、元组、字符串。
- 另外，Python提供了字典和集合这样的数据结构，它们属于无顺序的数据集合体，不能通过位置索引号来访问数据元素。

序列数据结构

- 通用的序列操作

操作	描述
X1+X2	联接序列X1和X2,生成新序列
X*n	序列X重复n次，生成新序列
X[i]	引用序列X中下标为i的成员
X[i：j]	引用序列X中下标为i到j-1的子序列
X[i：j：k]	引用序列X中下标为i到j-1的子序列，步长为k
len(X)	计算序列X中成员的个数
max(X)	序列X中的最大值
min(X)	序列X中的最小值
v in X	检查v是否在序列X中，返回布尔值
v not in X	检查v是否不在序列X中，返回布尔值

访问单个数据

- 用[ ]来访问序列中的一个元素。

比如访问字符串中的某个字符：

```
prompt = 'hello'
print(prompt[0])
```

输出：

```
h
```

```
print(prompt[4])
```

输出：

```
o
```

访问单个数据（续）

- 假设序列中的元素个数是n，下标的有效范围是0到n-1，或者-1到-n。
- 如果下标的绝对值大于n-1，则会发生下标越界错误。
- 如果下标的值为负数，表示从序列的最后一个元素往前引用，比如：

```
prompt = 'hello'
print(prompt[-1],prompt[-4])
```

输出

```
o e
```

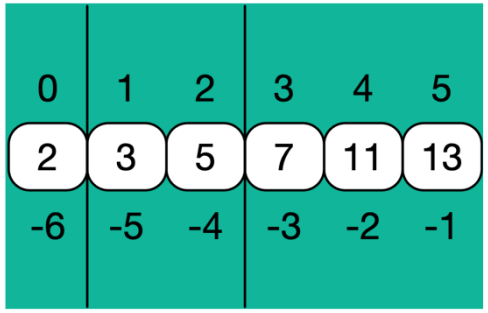
访问一部分数据

- 如果要访问序列中的一部分元素，可以使用切片（slice）。切片通过冒号分隔两个下标来实现。比如访问列表中的一部分：

```
a = [2,3,5,7,11,13]
print(a[1:3])
```

输出

```
[3,5]
```



注：如右图所示，切片a[1:3] 表示包含从第1个下标（1）开始到第2个下标（3）前面的下标（2）为止的部分元素的子序列（列表）。

访问一部分数据（续）

- 切片使用负的下标访问

a[1:-3]

结果：[3,5]

- 切片省略第2个下标，表示从第1个下标的元素开始到最后一个元素的切片。

a[2:]

结果：[5, 7, 11, 13]

- 第1个下标为0时，可以省略。

a[:3]

结果：[2, 3, 5]

a[:-2]

结果：[2, 3, 5, 7]

访问一部分数据（续2）

- 切片使用第3个参数，该参数表示切片选择元素的步长。

a[0:5:2]

结果是: [2, 5, 11]

- 切片使用第3个参数为负数时，表示逆向取切片。

a[-1:0:-1]

结果是：[13, 11, 7, 5, 3]

a[::-1]

结果是：[13, 11, 7, 5, 3, 2]

复制一个序列

- 如果将一个序列变量赋值给另外一个变量，则这2个变量表达了同一个序列。

```
a = [2, 3, 5, 7, 11, 13]
b = a
b[0] = 1
print(a)
```

输出：

```
[1, 3, 5, 7, 11, 13]
```

- 如果希望2个变量各自拥有独立的序列，可使用切片。

```
a = [2, 3, 5, 7, 11, 13]
b = a[:]
```

注：a[:]表示从头到尾的整个序列“切”出来的序列

序列的运算符

- 加号（+）连接2个序列

  a = [2,3,5,7,11,13]
  b = [4,6,8,9,10,12]
  print(a + b)

  输出：

  [2, 3, 5, 7, 11, 13, 4, 6, 8, 9, 10, 12]

- 乘号（*）重复序列

  [4,0,4]*3

  结果是：[4, 0, 4, 4, 0, 4, 4, 0, 4]

序列的运算符（续）

- 检查数据是否在序列中（in）

a = [2,3,5,7,11,13]
print(3 in a)

输出结果：True

- 对于列表和字符串，in有所不同，比较下面例子：

[2,3] in [2,3,5,7,11,13]

结果是：False

[2,3] in [[2,3],5,7,11,13]

结果是：True

'e' in 'hello'

结果是：True

'he' in 'hello'

结果是：True

in可以检查某个字符串是否是另一个字符串的一部分。

计算序列的长度

- len()函数返回序列内部元素的个数。

  len([2,3,5,7])

  结果是：4

  len('hello world')

  结果是：11

计算序列的最小值、最大值

- min()和max()函数计算序列中的最小值和最大值

  min([2,3,5,7,11,13])

  结果是：2

  min('好好学习天天向上')

  结果是：

  '上'

  注：字符串的大小是按照其Unicode 编码来比较的。

字符串使用

- 字符串是一连串的字符，用英文单引号（'） 或英文双引号（ " ）括起来。

  'Python is the best.'

  "Programming is fun."

- 引号必须成对出现；如果字符串中包含了单引号或双引号，则要用另一种引号括起来。

  "It's amazing!"
  'He said, "You are so cool!'

字符串使用（续）

- 长字符串

  用3个引号（单引号或双引号）括起来的字符串可以包含多行字符串。

  '''This is a test
for multiple lines
of text.'''

  表示包含了2个换行符的字符串：

  'This is a test\nfor multiple lines\nof text.'

如果要在程序中用多行表示一个字符串，则可以在每行的结尾用反斜杠（\）结束。

  'hello \
  world'

  结果是：'hello world'

字符串使用（续2）

- 原始字符串

  在一个字符串字面量前加一个字符r，表示这个字符串是原始字符串，其中的\不被当作是转义字符前缀。

  r = r'hello\nworld'

  print(r)

  输出：hello\nworld

  相当于r = 'hello\\nworld'

字符串使用（续3）

- **字符串是不可修改**

字符串中的数据（字符）是不能修改的。

```
s='hello'
s[0]='k'                    #会得到错误
```

可以通过用新的字符串对变量重新赋值，表示新的字符串。

```
s='hello'
s='bye'
```

这样变量s表示字符串'bye'。

字符串常用方法或函数

字符串常用方法或函数	解释
S.title()	字符串S首字母大写
S.lower()，	字符串S变小写
S.upper()	字符串S变大写
S.strip(),S.rstrip(),lstrip()	删除前后空格，删除右空格，删除左空格
S.find(sub[,start[,end]])	在字符串S中查找sub子串首次出现的位置
S.replace(old,new)	在字符串S中用new子串替换old子串
S.join(X)	将序列X合并成字符串
S.split(sep=None)	将字符串S拆分成列表
S.count(sub[,start[,end]])	计算sub子串在字符串S中出现的次数

字符串常用方法或函数（续）

- 查找子串 find()
  - 在字符串中查找子串，返回第一次出现的位置下标（从0开始），如果找不到返回-1。

```
s = 'This is a test.'
print(s.find('is'))
```

输出：

2

```
s = 'This is a test.'
print(s.find('ok'))
```

输出：

-1

字符串常用方法或函数（续2）

```
s = 'This is a test.'
print(s.find('is',3))        #指定查找开始位置
```

输出：

5

```
s = 'This is a test.'
print(s.find('is',3,6)) #指定查找开始位置及终止位置
```

输出：

-1

字符串常用方法或函数（续3）

- 统计子串出现的次数count()

```
s = 'This is a test.'
print(s.count('is'))
```

输出：

```
2
```

字符串常用方法或函数（续4）

- 修改大小写
  - 函数title() 将字符串中每个单词的首字母变成大写字母。

```
name = 'john johnson'
print(name.title())
```

输出：

John Johnson

  - 函数upper() 将字符串中所有字母变成大写字母。
  - 函数lower() 将字符串中所有字母变成小写字母。

字符串常用方法或函数（续5）

- 删除两端的空格
  - 函数rstrip() 去掉字符串右边的空格。

```
name = "Python "
name.rstrip()
```

结果是：

'Python'

  - 函数lstrip() 去掉字符串左边的空格。
  - 函数strip() 去掉字符串左右两边的空格。

字符串常用方法或函数（续6）

- 替换字符串中的子串 replace()

```
s = 'This is a test.'
t = s.replace('is', 'eez')
print(t)
```

输出：

Theez eez a test.

将数字转换成字符串

将数字类型的数据（整数、浮点数和复数）转换成字符串，有下列三种方法。

1. 函数str()

```
age = 23
print('Happy Birthday '+ str(age) +'!')
```

输出：

```
Happy Birthday 23!
```

注:不能写成：print('Happy Birthday '+ age +'!')

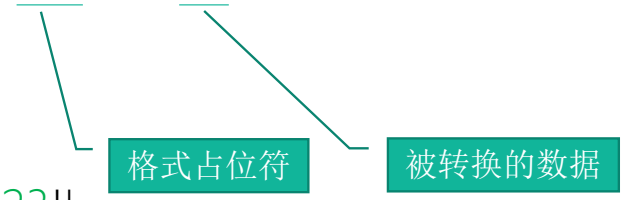
将数字转换成字符串（续）

2. 字符串格式化运算符 %

<带有转换说明符%的字符串> % (<需要转换的值>[,<需要转换的值>])

'Happy Birthday %d!' % (23)

结果是:



'Happy Birthday 23!'

也可以有多个转换的值

'Happy Birthday %d, %s!' % (23, 'John Johnson')

结果是:

'Happy Birthday 23, John Johnson!'

将数字转换成字符串（续2）

Python字符串的格式占位符

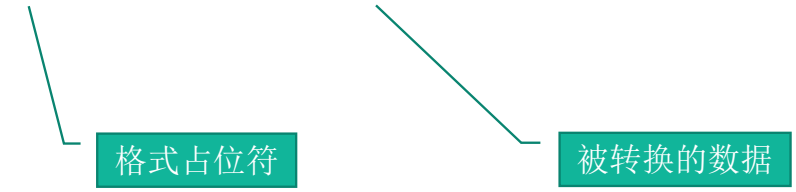
占位符	含义
%c	单个字符，替换成只有一个字符的字符串或将表示一个字符的Unicode码转成一个字符替换进来
%s	字符串
%d	整数
%u	无符号整数
%o	八进制数
%x	十六进制数
%X	字母大写的十六进制数
%f	浮点数
%e	科学计数法表示的浮点数
%E	大写的E表示的科学计数法
%g	综合的%f和%e，系统自动决定是否使用科学计数法
%G	大写表示的%g

将数字转换成字符串（续3）

3. format()函数

是字符串的一个函数，也是用来形成格式化的字符串。使用{}来表示占位符。

```
age = 23
'My age is {}'.format(age)
```



结果是：

```
'My age is 23'
```

将数字转换成字符串（续4）

- format()函数支持多个占位符，可以为占位符指定的被转换数据的索引。

  'my name is {} ,age {}'.format('Mary',18)
  结果是：

  'my name is Mary ,age 18'

  'my name is {1} ,age {0}'.format(10,'Mary')
  结果是：

  'my name is Mary ,age 10'

将数字转换成字符串（续5）

- format()函数也可以指定填充、对齐和宽度，以及精度和进制。它的一般格式是：

{<索引>:<填充字符><对齐方式><宽度.精度><格式>}

例：

```
'{0:*>10}'.format(10)   ##右对齐
'{0:*<10}'.format(10)   ##左对齐
'{0:*^10}'.format(10)   ##居中对齐
'{0:.2f}'.format(1/3)
'{0:b}'.format(10)      #二进制
'{0:o}'.format(10)      #八进制
'{0:x}'.format(10)      #16进制
'{:,}'.format(12345678901)  #千分位格式化
```

```
'********10'
'10********'
'****10****'
'0.33'
'1010'
'12'
'a'
'12,345,678,901'
```

输入四个字符串，求这些字符串的最大长度

```
length=0
for i in range(4):
    a=len(input())
    if a>length:
        length=a
print(length)
```

列表

- 列表（list）是最常用的Python数据类型，列表的数据项不需要具有相同的类型。列表类似其他语言的数组，但功能比数组强大的多。
- 创建一个列表，只要把逗号分隔的不同的数据项使用方括号括起来即可。实例如下：
- list1 = ['中国', '美国', 1997, 2000];
- list2 = [1, 2, 3, 4, 5];
- 列表索引从0开始。列表可以进行截取（切片）、组合等。
- 列表（list）是一个有序的序列结构，序列中的元素可以是不同的数据类型
- 列表可以进行一系列序列操作，如索引、切片、加、乘和检查成员等

列表创建

- 将列表中的各元素用逗号分隔开，并用中括号将所有元素包裹起来

```
>>> match_list=[2,'Jump Shot','Los Angeles Laker']

>>> print(match_list)

[2, 'Jump Shot', 'Los Angeles Laker']
>>> print(type(match_list))

<class 'list'>
>>>
```

列表

- 1. 访问列表中的值
- list1 = ['中国', '美国', 1997, 2000];
- list2 = [1, 2, 3, 4, 5, 6, 7];
- print ("list1[0]: ", list1[0])
- print ("list2[1:5]: ", list2[1:5])

列表

2．更新列表

可以对列表的数据项进行修改或更新，实例如下：

```
list = ['中国', 'chemistry', 1997, 2000];
print ( "Value available at index 2 : ")
print (list[2] )
list[2] = 2001;
print ( "New value available at index 2 : ")
print (list[2] )
```

3．删除列表元素

方法一：使用 del 语句来删除列表的的元素

方法二：使用 remove()方法来删除列表的元素

方法三：使用pop()方法方法来删除列表的指定位置的元素，无参数时删除最后一个元素

列表

4．添加列表元素

可以使用 append()方法在列表末尾添加元素，如下实例：

```
list1 = ['中国', '美国', 1997, 2000]
list1.append(2003)
print (list1)
```

以上实例输出结果：

```
['中国', '美国', 1997, 2000, 2003]
```

列表对象的增减

- 使用append()方法添加元素，该方法会在列表末尾位置添加数据元素
- 使用remove()方法删除元素

```
>>> match_list.append('POR')

>>> match_list

[2, 'Jump Shot', 'Los Angeles Laker', 'POR']
>>> match_list.remove(2)

>>> match_list

['Jump Shot', 'Los Angeles Laker', 'POR']
>>>
```

列表对象的增减

- remove()方法适用于知道要删除的值的情况，当我们不知道具体元素值，但是知道元素的索引位置时，我们可以使用 del 函数配合列表索引，删除索引位置的元素或者使用 pop()方法

```
match_list = [2,'Jump Shot', 'Los Angeles Lakers ', 'POR' ]
del match_list[-2]
print(match_list)
match_list = [2，'Jump Shot', 'Los Angeles Lakers ' ,'POR']
match_list.pop(-2)
print(match_list)
```

```
[2, 'Jump Shot', 'POR']
[2, 'Jump Shot', 'POR']
```

列表对象的增减

- 通过 insert() 方法在指定的索引位置添加数据元素

```
[2, 'Jump Shot', 'POR']
```

```
match_list.insert( 1, 'Los Angeles Lakers ' )
print(match_list)
```

```
[2, 'Los Angeles Lakers', 'Jump Shot', 'POR']
```

实践练习

```
match_list = [2, 'Los Angeles Lakers', 'Jump Shot', 'POR']
match_list.append('Left Side(L)')
match_list.append('2000-10-31')
del match_list[0]
match_list.insert(1,'LAL @ POR')
print(match_list)
```

- match_list = [2, 'Los Angeles Lakers', 'Jump Shot', 'POR']
- 现在需要添加更多关于某次投篮的信息，包括投篮位置（按区域划分）、比赛日期和对阵形式（主场或者客场）
  - 在match_list 插入 ‘Left Side(L)’，‘2000-10-31’，和'LAL @ POR'
  - 并删除数据元素2

列表索引

- Python语言中所有的索引都是从 0 开始计数的，如果列表中有 n 个元素，那么最后一个元素的索引是 n-1

```
['Jump Shot', 'Los Angeles Lakers', 'POR']
```

表1 kobe_list索引位置及相应的值

index（索引位置）	0	1	2	3
value（值）	2	'Jump Shot'	'Los Angeles Lakers'	'POR'

- 如果我们想要获取 match_list 中第 3 个元素及倒数第二个元素

match_list[2]

```
'Los Angeles Lakers'
```

match_list[-2]

```
'Los Angeles Lakers'
```

列表长度

- Python内置的用于判断列表长度的函数为 len()

```
match_list = [ 'Los Angeles Lakers '，'LAL @ POR'， '.Jump Shot '，'POR'，'Left Side(L)'， 12000-10-31']
len( match_list)
```

```
6
```

列表切片

- 切片操作需要提供起始索引位置和最后索引位置，然后用冒号：将两者分开

列表名称[起始索引位置：最后索引位置：步长]

- 如果未输入步长，则默认步长为 1
- 切片操作返回一系列从起始索引位置开始到最后索引位置结束的数据元素
- 需要注意的是，起始索引位置的值包含在返回结果中，而最后索引位置的值不包含在返回结果中

列表切片

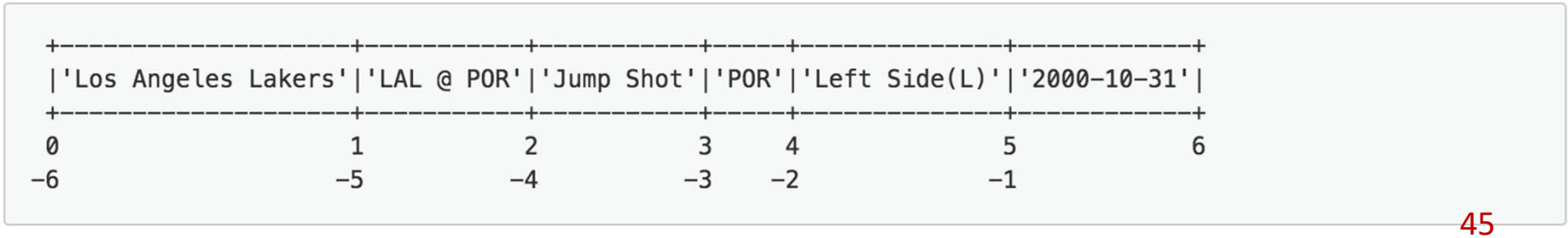
- 切片操作需要提供起始索引位置和最后索引位置，然后用冒号：将两者分开

```
>>> match_list=['Los Angeles Lakers','LAL @ POR','Jump Shot','POR','Left Side(L)','2000-10-31']

>>> len(match_list)

6
>>> print(match_list[1:4])

['LAL @ POR', 'Jump Shot', 'POR']
>>>
```



列表切片

```
+-------------------+----------+----------+-----+-------------+------------+
|'Los Angeles Lakers'|'LAL @ POR'|'Jump Shot'|'POR'|'Left Side(L)'|'2000-10-31'|
+-------------------+----------+----------+-----+-------------+------------+
 0                   1          2          3    4             5            6
-6                  -5         -4         -3   -2            -1
```

- 逆向切片

```
>>> print(match_list[-5:-2])

['LAL @ POR', 'Jump Shot', 'POR']
>>>
```

- 我们可以省略起始索引位置，表示从最开始进行切片，当我们将两个索引都省略之后，我们将按原样复制一个列表，如果想要将列表的顺序颠倒，则可以使用::-1

```
print(match_list[:-2])
print(match_list[:])
print(match_list[::-1])
```

```
['Los Angeles Lakers', 'LAL @ POR', 'Jump Shot', 'POR']
['Los Angeles Lakers', 'LAL @ POR', 'Jump Shot', 'POR', 'Left Side(L)', '2000-10-31']
['2000-10-31', 'Left Side(L)', 'POR', 'Jump Shot', 'LAL @ POR', 'Los Angeles Lakers']
```

嵌套列表

- 列表中的元素也可以是列表，这样可以将列表看成更高维的数组

```
nested_match_list = [['Los Angeles Lakers ' , 'LAL @ POR' ] , ' Jump Shot']
print(nested_match_list[0][0])
```

```
Los Angeles Lakers
```

- 拆开很容易理解

```
team_info = [ 'Los Angeles Lakers ' , ' LAL @ POR" ]
nested_match_list = [team_info, ' Jump Shot ']
print(nested_match_list[0][0])
print(team_info[0])
```

```
Los Angeles Lakers
Los Angeles Lakers
```

实践练习

```
match_list = ['Los Angeles Lakers', 'LAL @ POR', 'Jump Shot', 'POR', 'Left Side(L)',
'2000-10-31']
three_five = match_list[3:6]
year = three_five[-1][0:4]
print (three_five)
print (year)
```

- 现有某次投篮信息的列表match_list
- match_list = ['Los Angeles Lakers', 'LAL @ POR', 'Jump Shot', 'POR', 'Left Side(L)', '2000-10-31']
- 请使用切片方法取出索引位置3到5的元素，并存入到列表three_five中
- 并提取比赛日期中的年份存入变量year中
- 提示:
- 读取年份时，先使用索引方法将match_list或者three_five最后一个日期元素值'2000-10-13'提取出来，索引位置为-1，然后对该字符串进行切片操作，切片为0:4

Python列表内置函数和方法

表 2-13　Python 列表的方法和内置函数

方法	功能
list.append(obj)	在列表末尾添加新的对象
list.count(obj)	统计某个元素在列表中出现的次数
list.extend(seq)	在列表末尾一次性追加另一个序列中的多个值（用新列表扩展原来的列表）
list.index(obj)	从列表中找出某个值第一个匹配项的索引位置
list.insert(index, obj)	将对象插入列表
list.pop(index)	移除列表中的一个元素（默认最后一个元素），并且返回该元素的值
list.remove(obj)	移除列表中某个值的第一个匹配项
list.reverse()	反转列表中元素顺序
list.sort([func])	对原列表进行排序
len(list)	内置函数，列表元素个数
max(list)	内置函数，返回列表元素最大值
min(list)	内置函数，返回列表元素最小值
list(seq)	内置函数，将元组转换为列表

元组

Python的元组（tuple）与列表类似，不同之处在于元组的元素不能修改。元组使用小括号()，列表使用方括号[]。元组中的元素类型也可以不相同。

1．创建元组

```
tup1 = ('中国', '美国', 1997, 2000)
tup2 = (1, 2, 3, 4, 5 )
tup3 = "a", "b", "c", "d"
```

2．访问元组

```
tup1 = ('中国', '美国', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7 )
print ("tup1[0]: ", tup1[0])          # 输出元组的第一个元素
print ("tup2[1:5]: ", tup2[1:5])    # 切片，输出从第二个元素开始到第五个元素
print (tup2[2:])
```

元组

3．元组连接

元组中的元素值是不允许修改的，但可以对元组进行连接组合，实例如下：

```
tup1 = (12, 34,56)
tup2 = (78, 90)
#tup1[0] = 100                          #修改元组元素操作是非法的。
tup3 = tup1 + tup2          #连接元组，创建一个新的元组
print (tup3)
```

以上实例输出结果：(12, 34,56, 78, 90)

4．删除元组

元组中的元素值是不允许删除的，但可以使用del语句来删除整个元组。

元组

```
(2, 'Jump Shot', 'Los Angeles Lakers', 'POR')
```

- 元组（tuple）数据结构与列表类似，其中元素可以有不同的类型
- 但是元组中的元素是不可变的，即一旦初始化之后，就不能够再做修改（报错：元组对象不支持赋值）

```
match_list = [2, 'Jump Shot ' , 'Los Angeles Lakers ' , 'POR']
match_list[0] = 3
print(match_list)
match_tuple = (2,'Jump Shot', 'Los Angeles Lakers ', 'POR ')
match_tuple[0] = 3
```

```
[3, 'Jump Shot', 'Los Angeles Lakers', 'POR']
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-3-df8bbc49f24e> in <module>
      3 print(match_list)
      4 match_tuple = (2,'Jump Shot', 'Los Angeles Lakers ', 'POR ')
----> 5 match_tuple[0] = 3

TypeError: 'tuple' object does not support item assignment
```

```
(2, 'Jump Shot', 'Los Angeles Lakers', 'POR')
```

- 由于元组是不可变的，因此元组对象没有append()、insert()和del这样的方法。
- 实际上，tuple的使用可以使得代码更安全，防止错误赋值导致重要对象的改变。

```
match_list = [2, 'Jump Shot ' , 'Los Angeles Lakers ' , 'POR']
match_list[0] = 3
print(match_list)
match_tuple = (2,'Jump Shot', 'Los Angeles Lakers ', 'POR ')
match_tuple[0] = 3
```

```
[3, 'Jump Shot', 'Los Angeles Lakers', 'POR']
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-3-df8bbc49f24e> in <module>
      3 print(match_list)
      4 match_tuple = (2,'Jump Shot', 'Los Angeles Lakers ', 'POR ')
----> 5 match_tuple[0] = 3

TypeError: 'tuple' object does not support item assignment
```

实践练习

```
match_tuple = ([2, 'Jump Shot'], 'Los Angeles Lakers', 'POR')
match_tuple[0][0] = 3
match_tuple[0][1] = 'Slam Dunk Shot'
print(match_tuple)
```

- 现有投篮信息的元组match_tuple

match_tuple = ([2, 'Jump Shot'], 'Los Angeles Lakers', 'POR')

- 请将match_tuple中的投篮ID由2改为3，投篮类型由Jump Shot改为'Slam Dunk Shot'
- 提示：通过match_tuple[][]方式访问元组中的列表元素中的某一个元素

元组与列表转换

可以使用下面方法将元祖转换为列表：

列表对象=list(元组对象)

```
tup=(1, 2, 3, 4, 5)
list1= list(tup)                                    #元组转为列表
print (list1)                                       #返回[1, 2, 3, 4, 5]
```

可以使用下面方法将列表转换为元祖：

元组对象= tuple (列表对象)

- nums=[1, 3, 5, 7, 8, 13, 20]
- print (tuple(nums))　　列表转为元组，返回(1, 3, 5, 7, 8, 13, 20)

字典

- Python字典（dict）是一种可变容器模型，且可存储任意类型对象，如字符串、数字、元组等。字典也被称作关联数组或哈希表。
- 1. **创建字典**
- **键值对key-value**
- 一个简单的字典实例：

```
dict = {'zhangsan' : 91 , 'wangwu' : 80}
```

- 也可如此创建字典：

```
dict1 = { 'abc': 456 }
dict2 = { 'abc': 123, 98.6: 37 }
```

字典

2．访问字典里的值

访问字典里的值时把相应的键放入方括号里，如下实例：

```
dict = {'Name': '王海', 'Age': 17, 'Class': '计算机一班'}
print ("dict['Name']: ", dict['Name'])
print ("dict['Age']: ", dict['Age'])
```

3．修改字典

```
dict = {'Name': '王海', 'Age': 17, 'Class': '计算机一班'}
dict['Age'] = 18                              # 更新键/值对（update existing entry）
dict['School'] = "中原工学院" # 增加新的键/值对（add new entry）
print ("dict['Age']: ", dict['Age'] )
print ( "dict['School']: ", dict['School'];
```

字典

```
{1: 'Jump Shot',
 2: 'Jump Shot',
 3: 'Jump Shot',
 4: 'Jump Shot',
 5: 'Driving Dunk Shot',
 6: 'Jump Shot'}
```

- 字典（dict）在其他语言中被称作哈希映射（hash map）或者相关数组（associative arrays）
- 字典是一种大小可变的键值对集，其中的键（key）和值（value）都是Python对象
- 字典用在需要高速查找的地方

字典创建

- 字典的创建使用大括号 {} 包含键值对，并用冒号 : 分隔键和值，形成 **键:值** 对

```
match_dict = {2: 'Jump Shot '， 1 : 'Jump Shot', 3: ' Jump Shot ' , 4: "Jump Shot'， 5: 'Driving Dunk Shot '， 6: 'Jump Shot'}
match_dict
```

```
{1: 'Jump Shot',
 2: 'Jump Shot',
 3: 'Jump Shot',
 4: 'Jump Shot',
 5: 'Driving Dunk Shot',
 6: 'Jump Shot'}
```

- 可以看出，字典中的数据元素是无序的，并不会按照初始化的顺序排列。不同键所对应的值可以相同，但是字典中的键必须是唯一的

字典创建

- 利用for循环和zip()函数创建字典
- zip()函数用于将多个序列（列表、元组等）中的元素配对，产生一个如 [(列表1元素,列表2元素),(,)] 的新的元组列表；for循环用于重复执行将值放入键中的操作。

```
shot_id=[1,2,3]
shot_zone_area=['Right Side(R)','Left Side(L)','Left Side Center(LC)']
match_dict={}
for key,value in zip(shot_id,shot_zone_area):
    match_dict[key]=value
print(match_dict)
```

{1: 'Right Side(R)', 2: 'Left Side(L)', 3: 'Left Side Center(LC)'}

实践练习

```
shot_id = [1,2,3]
shot_zone_area = ['Right Side(R)','Left Side(L)','Left Side Center(LC)']
match_dict = {1:'Right Side(R)',2:'Left Side(L)',3:'Left Side Center(LC)'}
print(match_dict)
```

- 请创建关于投篮信息的字典match_dict，键为shot_id，值为shot_zone_area
- 其中列表shot_id与列表shot_zone_area应按照顺序一一对应
- shot_id = [1,2,3]
- shot_zone_area = ['Right Side(R)','Left Side(L)','Left Side Center(LC)']

字典索引

- 字典的元素访问（以及插入、设置）方式与列表和元组一样。不同的是，列表和元组的索引号是按照顺序自动生成，而字典的索引号是键

```
match_dict={1:'Jump Shot',2:'Jump Shot',3:'Jump Shot',4:'Jump Shot',5:'Driving Dunk Shot',6:'Jump Shot',7:'Jump Shot'}
print(match_dict[5])
```

```
Driving Dunk Shot
```

- 字典中某值的索引还可以通过 get 方法，如果字典不包含某个键，可以返回 None ，或者自己指定的值

```
>>> match_dict.get(1)
'Jump Shot‘
>>> match_dict.get(6)
'Jump Shot'
>>>
```

字典索引

- 如果在字典中不存在索引的键，则系统会报错

match_dict[8]

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-6-8a84ce4780f0> in <module>
      1 match_dict={1:'Jump Shot',2:'Jump Shot',3:'Jump Shot',4:'Jump Shot',5:'Driving Dunk Shot',6:'Jump Shot',7:'Jump Sho
t'}
----> 2 match_dict[8]

KeyError: 8
```

字典索引

- 我们可以通过 in 判断是否存在某个键，其语法跟在列表和元组中判断是否存在某个值是相同的，也可以使用内置的 has_key() 方法

```
print(5 in match_dict)
print(8 in match_dict)
print(match_dict.has_key(5))
print(match_dict.has_key (8))
```

```
True
False
True
False
```

字典索引

- 如果不太确定字典中有哪些键或者值，我们可以使用 keys() 方法或者values()方法

```
match_dict={1:'Jump Shot',2:'Jump Shot',3:'Jump Shot',4:'Jump Shot',5:'Driving Dunk Shot',6:'Jump Shot',7:'Jump Shot'}
print('keys are',match_dict.keys())
print('values are',match_dict.values())
```

```
keys are dict_keys([1, 2, 3, 4, 5, 6, 7])
values are dict_values(['Jump Shot', 'Jump Shot', 'Jump Shot', 'Jump Shot', 'Driving Dunk Shot', 'Jump Shot', 'Jump Shot'])
```

```
match_dict={1:'Jump Shot',2:'Jump Shot',3:'Jump Shot',4:'Jump Shot',5:'Driving Dunk Shot',6:'Jump Shot',7:'Jump Shot'}
for i in match_dict.keys():
    print(i,end=‘ ‘)
```

```
1 2 3 4 5 6 7
```

字典索引

- 在有些情况下，我们需要取出字典中的键值对用于下一步的分析，此时可以使用 items() 方法，该方法将返回所有键值对，并将其保存在一个元组列表（列表中的元素为元组）中

match_dict.items( )

```
[(1, 'Jump Shot'),
 (2, 'Jump Shot'),
 (3, 'Jump Shot'),
 (4, 'Jump Shot'),
 (5, 'Driving Dunk Shot'),
 (6, 'Jump Shot'),
 (7, 'Jump Shot')]
```

字典元素删减

- 字典的删减有三种方法
  - 使用 del 函数对单一元素或者整个字典进行删除
  - 使用 pop() 方法删除单一元素
  - 使用 clear() 方法清空词典的所有元素

```
del match_dict [1]
print(match_dict)
match_dict.pop(2)
print(match_dict)
match_dict.clear( )
print(match_dict)
```

```
{2: 'Jump Shot', 3: 'Jump Shot', 4: 'Jump Shot', 5: 'Driving Dunk Shot', 6: 'Jump Shot', 7: 'Jump Shot'}
{3: 'Jump Shot', 4: 'Jump Shot', 5: 'Driving Dunk Shot', 6: 'Jump Shot', 7: 'Jump Shot'}
{}
```

集合

```
set(['Driving Dunk Shot', 'Running Jump Shot', 'Jump Shot', 'Driving Layup Shot', 'Layup Shot'])
```

- 集合（set）是一种无序集，它是一组键的集合，不存储值
- 在集合中，重复的键是不被允许的。集合可以用于去除重复值
- 集合也可以进行数学集合运算，如并、交、差以及对称差等。
- 应用：

  去重。把一个列表变成集合，就自动去重了

  关系测试。测试两组数据之前的交集、差集、并集等关系

集合

集合（set）是一个无序不重复元素的序列。集合基本功能是进行成员关系测试和删除重复元素。

1. 创建集合

可以使用大括号({})或者 set()函数创建集合，注意：创建一个空集合必须用 set() 而不是 { }，因为 { } 是用来创建一个空字典。

```
student = {'Tom', 'Jim', 'Mary', 'Tom', 'Jack', 'Rose'}
print(student)   # 输出集合，重复的元素被自动去掉


shot_type_set = set( )
```

集合

可以使用“-”、“|”、“&”运算符进行集合的差集、并集、交集运算。

```
# set可以进行集合运算
a = set('abcd')        #a= {'a', 'b', 'c', 'd' }
b = set('cdef')
print(a)
print("a和b的差集：", a - b)     # a和b的差集{'a', 'b' }
print("a和b的并集：", a | b)     # a和b的并集 {'a', 'b', 'c', 'd' , 'e' }
print("a和b的交集：", a & b)     # a和b的交集{'c', 'd' }
print("a和b中不同时存在的元素：", a ^ b)     # a和b中不同时存在的元素
```

集合运算

- 集合支持数学集合运算，如并、交、差以及对称差等

```
shot_type_set1 = {'Jump Shot', 'Running Jump Shot', 'Driving Dunk Shot'}
shot_type_set2 = {'Layup Shot', 'Jump Shot', 'Driving Layup Shot'}
print '\t',shot_type_set1
print '\t',shot_type_set2
print '并集（或）','\n\t',shot_type_set1|shot_type_set2
print '交集（与）','\n\t',shot_type_set1&shot_type_set2
print '差集','\n\t',shot_type_set1-shot_type_set2
print '对称差（异或）','\n\t',shot_type_set1^shot_type_set2
```

```
  set(['Driving Dunk Shot', 'Running Jump Shot', 'Jump Shot'])
  set(['Jump Shot', 'Driving Layup Shot', 'Layup Shot'])
并集（或）
  set(['Running Jump Shot', 'Jump Shot', 'Driving Dunk Shot', 'Driving Layup Shot', 'Layup Shot'])
交集（与）
  set(['Jump Shot'])
差集
  set(['Driving Dunk Shot', 'Running Jump Shot'])
对称差（异或）
  set(['Driving Dunk Shot', 'Running Jump Shot', 'Driving Layup Shot', 'Layup Shot'])
```

集合运算

- 集合支持数学集合运算，如并、交、差以及对称差等

表2 Python的集合运算

函数	其他表示法	说明
a.add(x)	N/A	把元素x添加到集合a中
a.remove(x)	N/A	把元素x从集合a中删除
a.union(b)	a | b	a和b中全部的唯一元素
a.intersection(b)	a & b	a和b都有的元素
a.difference(b)	a - b	a中不属于b的元素
a.symmetric_difference(b)	a ^ b	a或b中不同时属于a和b的元素
a.issubset(b)	N/A	如果a的全部元素都包含于b，则为True
a.issuperset(b)	N/A	如果b的全部元素都包含于a，则为True
a.isdisjoint(b)	N/A	如果a和b没有公共元素，则为True

总结

- 列表、元组、字典、集合
- 创建、索引、增减、切片操作